TweetPulse: Task List by Member

Nov 4, 2026 · @Ayesha

This list follows Section 7 (Division of Work) and Section 8 (Milestones) of your proposal. Tick items off as you finish them, and commit each piece from your own GitHub account. Code for every file is in the Full Final Project Manual.

Everyone (shared tasks)

- [ ] Commit regularly from your own GitHub account, with specific messages
- [ ] Be able to explain every line of your own files without notes (viva)
- [ ] Write the report sections for your own modules (8-9 pages in total)
- [ ] Cite all external code, datasets and AI assistance in the report
- [ ] Run the project from a fresh clone on another machine, using only the README
- [ ] Check that no API key, password, venv, `__pycache__` or large dataset is in the repository or its history
- [ ] Agree who writes which ethics point (suggestion: Ayesha on misuse and limits, M. Hamdan on slang and language bias, Mohsina on privacy and dataset bias)

Ayesha Nehal (Leader): models, evaluation, menu, integration

Repository setup (Week 1)

- [ ] Create the public GitHub repository and add both members as collaborators
- [ ] Add `.gitignore`, the folder skeleton (`src/`, `data/raw`, `data/processed`, `outputs/figures`, `outputs/results`, `docs/`) and `.gitkeep` files
- [ ] Push the first commit and confirm every member has pushed one
- [ ] Put the proposal (PDF and MD) in `docs/`

models.py (Labs 05, 10-11)

- [ ] `Tweet` class: constructor that rejects empty text, private attributes, properties, `__str__`
- [ ] `SentimentClassifier` abstract base class with abstract `train` and `predict`, plus `predict_many`
- [ ] `VaderClassifier` (reads raw text, maps compound score to a label)

- [ ] `NaiveBayesClassifier` with Bag-of-Words or TF-IDF
- [ ] `LogisticClassifier` with unigrams or unigrams plus bigrams
- [ ] Docstrings on every class and method

evaluate.py (Labs 10-11)

- [ ] One stratified `train_test_split` with a fixed `random_state`, shared by all models
- [ ] Accuracy, macro precision, recall and F1, and the confusion matrix on the test set only
- [ ] Compare all five configurations in one table
- [ ] Score the best model once on `test.csv` and never fit or tune on it

main.py (Labs 01-02)

- [ ] `get_int` with type conversion and re-prompting on bad input
- [ ] The 11-option menu loop and the state that passes data between options
- [ ] Clear messages when an option is run before its prerequisite

Integration, README and submission

- [ ] Merge teammates' modules and run options 1 to 11 end to end (Weeks 3-4)
- [ ] README: description, members with roll numbers, setup, run command `python src/main.py`, features, limitations, data links
- [ ] Generate `requirements.txt` with `pip freeze`
- [ ] Report: system design and class diagram, results table, confusion matrix, disagreement analysis, contribution table
- [ ] Confirm `docs/` holds all four documents (proposal and report, each as PDF and MD)
- [ ] Submit the repository link on Google Classroom, naming every member and roll number

M. Hamdan (Member 2): text processing, NumPy, visualisations

preprocessing.py (Labs 03, 04, 12)

- [ ] `ensure_nltk_resources` and `get_stopwords` (a set that keeps negations like not and never)
- [ ] `normalise_quotes` (backtick to apostrophe) and `expand_negations`
- [ ] `clean_text` with default parameters and a docstring, using a comprehension and `filter` with a lambda
- [ ] `add_clean_column` using `map` with a lambda

- [ ] `word_frequencies` (dict), `top_words` (`sorted` with a lambda key, returns tuples) and `build_vocabulary` (set)
- [ ] `nltk_demo` for tokens and POS tags
- [ ] Test on tweets with URLs, @mentions, hashtags, stretched words like soooo and didn`t, and keep before and after examples for the report

numpy_ops.py (Lab 07)

- [ ] `build_bow_matrix` (about 2,000 tweets by 500 words)
- [ ] `row_normalise` using broadcasting
- [ ] `top_words_for_class` using a boolean mask and `sum(axis=0)`
- [ ] Note the array shapes for the report and make sure there are no element loops

visualize.py (Lab 09)

- [ ] Five figures: class counts, tweet-length histogram, VADER score box plot, confusion matrix, model comparison
- [ ] Every figure has a title, axis labels and a legend where needed, and is saved to `outputs/figures/`
- [ ] After real results exist, write 2-3 interpreting sentences per figure

Report and viva

- [ ] Report sections: dataset and preprocessing, data structure justifications (list, tuple, dict, set), Labs 03, 04, 07, 09 and 12 coverage
- [ ] Be ready to state array shapes, justify each data structure and make a small live change (for example a new stopword)

Mohsina Zeeshan (Member 3): data layer, API and cache, Pandas

exceptions.py and data_loader.py (Lab 06)

- [ ] `DataLoadError` custom exception
- [ ] `load_tweets` with `encoding='latin-1'` and specific `except` blocks (missing file, empty file, malformed file, missing columns)
- [ ] `load_json`, `save_json` and `write_log` (CSV, JSON and TXT all used)
- [ ] `fetch_headlines`: `requests` call, status code checked, reply cached to `data/raw/api_cache.json`, fallback to cache offline

- [ ] Open the Hacker News endpoint in a browser first, run option 9 once with internet, and commit `api_cache.json`
- [ ] Put `train.csv` and `test.csv` in `data/raw/` (or a small sample if size is a problem) and document the download in the README

analysis.py (Lab 08)

- [ ] `clean_dataframe` with `dropna`, `drop_duplicates`, lowercase labels and a word-count column
- [ ] `summarise_by_sentiment` (`groupby`), `sentiment_by_group` (`crosstab`) and `filter_rows`
- [ ] `add_vader_scores`
- [ ] Check whether Country, Age and Time columns look synthetic and say so in the report

Robustness testing

- [ ] Missing data file, empty CSV, CSV without a `sentiment` column
- [ ] No network with a cache, and no network without a cache
- [ ] Record the observed results in the report's testing table

Report and viva

- [ ] Report sections: data source and size, Pandas results with real numbers, Labs 06 and 08 coverage, testing
- [ ] Be ready to show what happens with a missing file and with the network off, and to justify each specific exception

Week-by-week targets (from the proposal)

Week	Demonstrable goal	Owners
1	Repository and skeleton pushed, first commit from every member, `train.csv` loads, menu starts and exits	All
2	Cleaned DataFrame summarised, API cached and reloaded offline, `DataLoadError` handled, cleaner produces tokens	Mohsina, M. Hamdan
3	NumPy matrix shapes printed, three classifiers train, five-model table prints on one split	M. Hamdan, Ayesha

Week	Demonstrable goal	Owners
4	Five figures saved, full menu works with disagreement analysis, survives missing file and no network	All
5	README, report, PDF and MD exports in `docs/`, fresh-clone test, submit	All